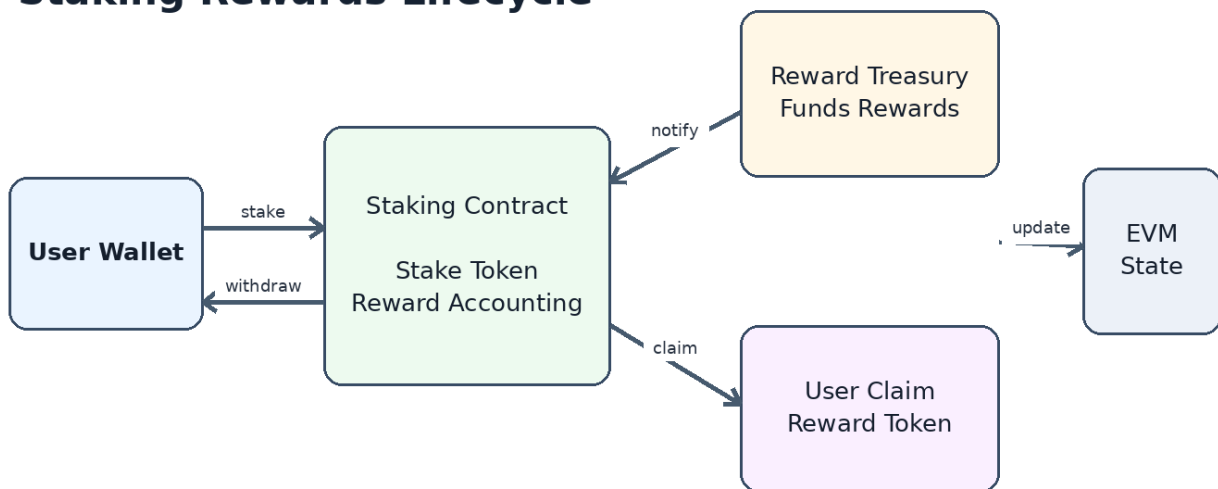


DAY 15

DeFi Development: Staking, Vesting, Lending, Vaults & Flash Loans

Complete 60-Minute Beginner-to-Freelancer Lecture

Staking Rewards Lifecycle



Goal: Move from using DeFi protocols to designing, testing and explaining the smart-contract patterns behind staking, vesting, lending, yield vaults and flash-loan workflows.

Course context: Day 14 explained DEXs, AMMs and liquidity pools. Day 15 builds the next layer: reward distribution, time-based token release, collateralized lending, tokenized vault shares and atomic borrowing.

1. Learning Objectives and 60-Minute Teaching Plan

By the end of Day 15, a beginner should be able to explain staking reward accounting, implement a safe staking/claim flow, describe token vesting and cliffs, explain collateralized lending and health factor, recognize ERC-4626 vault terminology, trace an Aave-style flash loan, evaluate arbitrage as a cost equation, and turn a DeFi client request into a testable implementation plan.

Time	Topic	Outcome
0-5 min	DeFi development map	Connect Day 14 swaps to staking, lending, vaults and atomic liquidity.
5-13 min	Staking fundamentals	Understand stake, withdraw, rewards and reward funding.
13-20 min	Reward accounting	Compare naive loops with scalable reward-per-token accounting.
20-26 min	Vesting	Understand cliffs, schedules, releasable amount and beneficiary design.
26-34 min	Lending/borrowing	Understand collateral, LTV, interest, health factor and liquidation risk.
34-40 min	Yield farming + ERC-4626	Understand assets, shares, deposits, redemptions and vault strategy.
40-48 min	Flash loans	Trace borrow -> callback -> logic -> repayment in one transaction.
48-53 min	Arbitrage economics	Calculate net profit after DEX, gas, price-impact and flash-loan costs.
53-58 min	Security + testing	Identify accounting, token, oracle and integration risks.
58-60 min	Freelancer lab + review	Translate a client request into architecture, tests and deliverables.

60-second target explanation: A DeFi developer manages asset accounting across time and across contracts. Staking distributes funded rewards to depositors, vesting releases assets according to time, lending uses collateral and risk parameters to support borrowing, vaults issue shares representing pooled assets, and flash loans provide atomic liquidity that must be repaid before the transaction finishes.

2. DeFi Development Map: Capital Moves Through Contracts

Day 14 focused on exchanging assets. Day 15 focuses on what happens when capital stays inside protocols for longer than one swap: it may earn rewards, become collateral, receive a share token, unlock over time, or be temporarily borrowed inside one atomic transaction.

Pattern	User gives protocol	Protocol gives user	Main developer problem
Staking	Stake token or position	Reward entitlement	Fair reward accounting over time
Vesting	Pre-funded token allocation	Gradual release	Correct time schedule and beneficiary rights
Lending	Liquidity or collateral	Interest / borrow capacity	Solvency, oracle and interest accounting
Yield vault	Underlying ERC-20 asset	Vault shares	Asset/share conversion and strategy accounting
Flash loan	Nothing up front; transaction promise	Temporary liquidity	Atomic repayment plus fee

2.1 A key mental model: assets, accounting and permissions

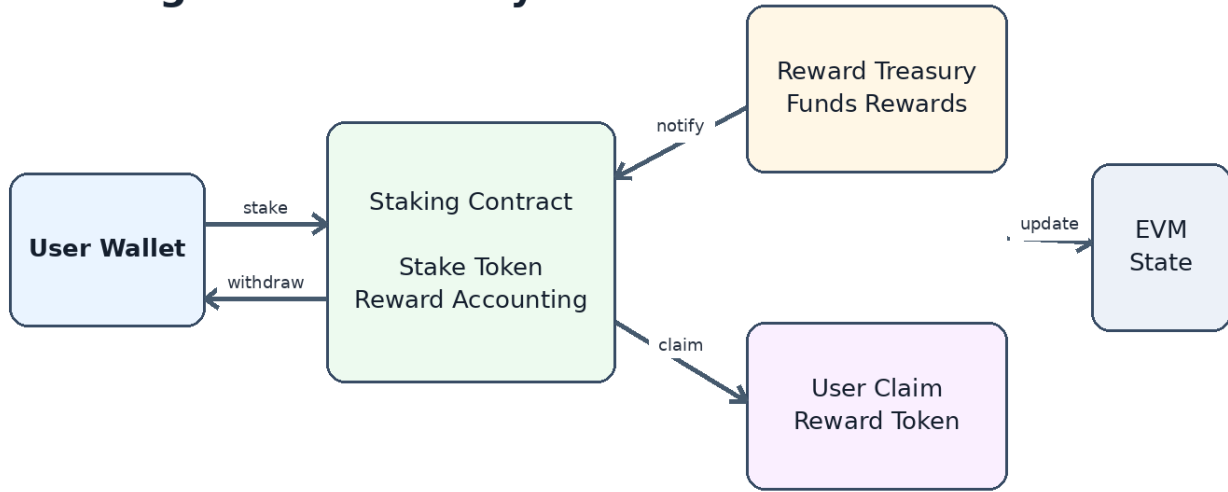
- Assets: which ERC-20/native tokens move and who owns them at each step?
- Accounting: what state proves a user balance, reward, debt, share or release amount?
- Permissions: who can fund rewards, change parameters, pause, upgrade or withdraw protocol-owned assets?
- External dependencies: token contracts, DEX pools, price oracles, lending pools and RPC infrastructure.

Freelancer habit: Before writing Solidity, draw the money flow. Mark every transfer, every state update, every privileged role and every external contract call. This catches many design errors before code exists.

3. Staking Fundamentals

Staking contracts commonly let users deposit an ERC-20 token and receive rewards while their stake remains deposited. In outsourcing work, "staking" may mean many different business models, so requirements must be clarified before coding.

Staking Rewards Lifecycle



Term	Meaning
Stake token	Asset deposited by the user.
Reward token	Asset paid as incentive; may equal the stake token or be different.
Reward rate	How quickly funded rewards are distributed.
Stake balance	Amount currently attributed to one user.
Total staked	Sum of all active stake balances.
Claim	Transfer accrued reward to the user.
Exit	Usually withdraw stake and claim rewards in one action.

3.1 Questions to ask a staking client

1. Is the reward token the same as the stake token?
2. Are rewards fixed, APR-like, block-based or time-based?
3. Where do reward tokens come from?
4. Can users withdraw at any time?
5. Is there a lock period or early-withdraw penalty?
6. Can the admin change reward parameters?
7. Should reward funding be capped by the tokens actually held?

4. Why Naive Reward Loops Do Not Scale

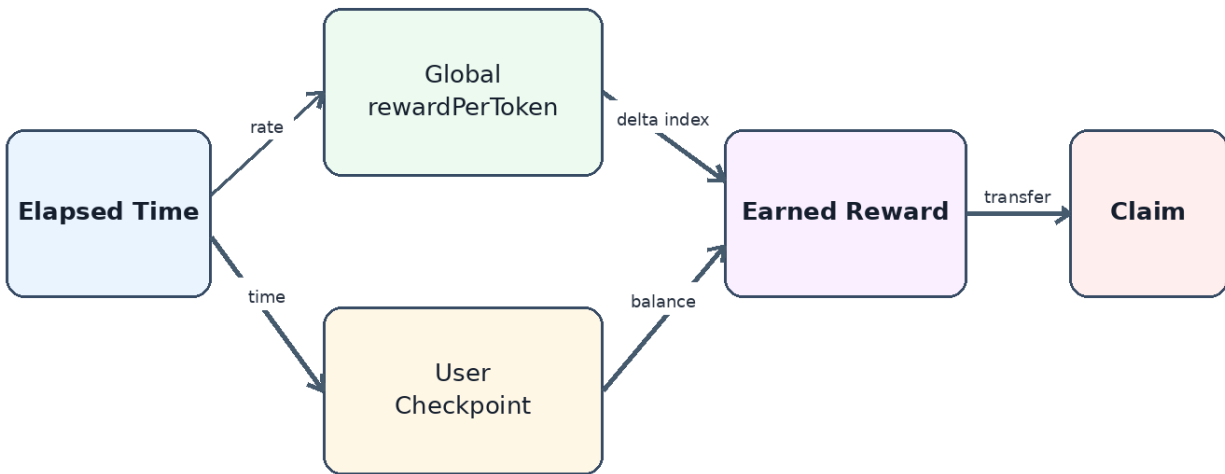
A beginner may try to distribute rewards by looping through every staker. That design becomes increasingly expensive as the user list grows. A scalable design updates global accounting and calculates each user's entitlement only when that user interacts.

```
// Anti-pattern idea: loop through everyone
for (uint256 i = 0; i < stakers.length; i++) {
    rewards[stakers[i]] += calculate(stakers[i]);
}
```

Problem: An unbounded loop makes gas cost depend on the number of users. At enough users, routine actions can become too expensive or impossible to execute within block gas constraints.

4.1 Reward-per-token accounting

Scalable Reward Accounting: Global Index + User Check



The common pattern maintains a global reward-per-token accumulator. Each user stores the accumulator value already accounted for. On stake, withdraw or claim, the contract updates the global index and then computes only that user's newly earned amount.

```
rewardPerToken +=
    elapsed * rewardRate * PRECISION / totalSupply;

userEarned +=
    userBalance * (rewardPerToken - userPaid) / PRECISION;

userPaid = rewardPerToken;
```

5. Staking Contract State and Update Pattern

The following variables form a useful professional mental model. Exact names differ among implementations, but the relationships are common.

State	Purpose
totalSupply	Total amount currently staked.
balanceOf[user]	Current stake belonging to a user.
rewardRate	Reward tokens emitted per second.
periodFinish	Timestamp when the current reward period ends.
lastUpdateTime	Last timestamp included in global reward accounting.
rewardPerTokenStored	Cumulative reward allocation per staked token.
userRewardPerTokenPaid[user]	Global accumulator already credited to the user.
rewards[user]	Accrued but unclaimed rewards.

5.1 The update modifier idea

```
modifier updateReward(address account) {
    rewardPerTokenStored = rewardPerToken();
    lastUpdateTime = lastTimeRewardApplicable();

    if (account != address(0)) {
        rewards[account] = earned(account);
        userRewardPerTokenPaid[account] = rewardPerTokenStored;
    }
    _;
}
```

Ordering: Update accounting before changing the user stake balance. Otherwise the new balance can incorrectly receive past rewards, or the old balance can miss rewards it earned.

6. Practical Contract - StakingRewards (Part 1)

This teaching example uses OpenZeppelin SafeERC20 and ReentrancyGuard. The owner funds a reward period, while users stake, withdraw and claim. In a production project you would add business-specific limits, pause/emergency processes, monitoring and a full test suite.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

contract StakingRewards is ReentrancyGuard, Ownable {
    using SafeERC20 for IERC20;

    IERC20 public immutable stakingToken;
    IERC20 public immutable rewardsToken;

    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;

    uint256 public rewardRate;
    uint256 public periodFinish;
    uint256 public lastUpdateTime;
    uint256 public rewardPerTokenStored;

    mapping(address => uint256) public userRewardPerTokenPaid;
    mapping(address => uint256) public rewards;

    uint256 private constant PRECISION = 1e18;

    constructor(IERC20 stake_, IERC20 reward_) Ownable(msg.sender) {
        stakingToken = stake_;
        rewardsToken = reward_;
    }
}
```

7. Practical Contract - StakingRewards (Part 2)

```
function lastTimeRewardApplicable() public view returns (uint256) {
    return block.timestamp < periodFinish ? block.timestamp : periodFinish;
}

function rewardPerToken() public view returns (uint256) {
    if (totalSupply == 0) return rewardPerTokenStored;

    return rewardPerTokenStored +
        ((lastTimeRewardApplicable() - lastUpdateTime)
         * rewardRate * PRECISION) / totalSupply;
}

function earned(address account) public view returns (uint256) {
    return rewards[account] +
        (balanceOf[account]
         * (rewardPerToken() - userRewardPerTokenPaid[account]))
        / PRECISION;
}

modifier updateReward(address account) {
    rewardPerTokenStored = rewardPerToken();
    lastUpdateTime = lastTimeRewardApplicable();

    if (account != address(0)) {
        rewards[account] = earned(account);
        userRewardPerTokenPaid[account] = rewardPerTokenStored;
    }
    _;
}
```

Instructor explanation

- `rewardPerToken()` is read-only math over elapsed time and current total stake.
- `earned(user)` combines already stored reward with the user's uncheckpointed share of the global accumulator.
- The modifier freezes the accounting point before the actual stake/withdraw/claim mutation.

8. Practical Contract - StakingRewards (Part 3)

```
function stake(uint256 amount)
    external
    nonReentrant
    updateReward(msg.sender)
{
    require(amount > 0, "amount=0");
    totalSupply += amount;
    balanceOf[msg.sender] += amount;
    stakingToken.safeTransferFrom(msg.sender, address(this), amount);
}

function withdraw(uint256 amount)
    public
    nonReentrant
    updateReward(msg.sender)
{
    require(amount > 0, "amount=0");
    require(balanceOf[msg.sender] >= amount, "insufficient stake");
    totalSupply -= amount;
    balanceOf[msg.sender] -= amount;
    stakingToken.safeTransfer(msg.sender, amount);
}

function getReward()
    public
    nonReentrant
    updateReward(msg.sender)
{
    uint256 reward = rewards[msg.sender];
    if (reward == 0) return;
    rewards[msg.sender] = 0;
    rewardsToken.safeTransfer(msg.sender, reward);
}

function exit() external {
    withdraw(balanceOf[msg.sender]);
    getReward();
}
```

Review point: This example updates internal state before token transfers and uses nonReentrant on transfer-bearing entry points. Still test unusual tokens and client-specific assumptions - SafeERC20 does not make every ERC-20 economic behavior safe.

9. Funding a Reward Period Safely

Reward funding is an admin operation and must not promise more tokens than the contract can pay. A typical design calculates the remaining rewards from an active period, combines them with the new funding amount, and sets a new rewardRate.

```
function notifyRewardAmount(uint256 reward, uint256 duration)
    external
    onlyOwner
    updateReward(address(0))
{
    require(duration > 0, "duration=0");

    rewardsToken.safeTransferFrom(msg.sender, address(this), reward);

    if (block.timestamp >= periodFinish) {
        rewardRate = reward / duration;
    } else {
        uint256 remaining = periodFinish - block.timestamp;
        uint256 leftover = remaining * rewardRate;
        rewardRate = (reward + leftover) / duration;
    }

    lastUpdateTime = block.timestamp;
    periodFinish = block.timestamp + duration;
}
```

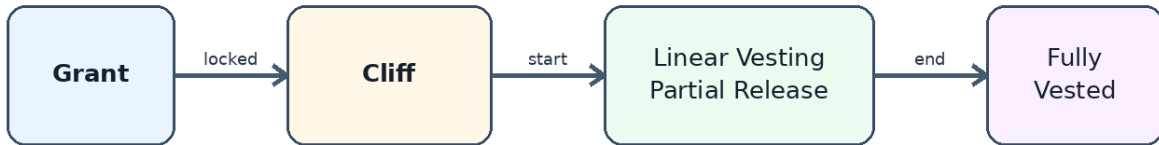
9.1 Questions for production design

- Should reward duration be fixed by the contract rather than supplied by the owner?
- Should there be an upper bound on rewardRate?
- What happens if stakeToken and rewardsToken are the same token?
- Can the admin recover unrelated tokens without touching user stake/rewards?
- What event should be emitted for reward funding and parameter changes?

10. Token Vesting: Time-Based Release

Vesting solves a different problem from staking. Staking generally rewards an active deposited position. Vesting starts with an allocation and restricts when the beneficiary can release it.

Token Vesting Timeline



Concept	Meaning
Start	Timestamp when vesting schedule begins.
Cliff	Optional initial period with no release.
Duration	Time from start until fully vested.
Vested amount	Amount that should be unlocked at a timestamp.
Released amount	Amount already withdrawn by beneficiary.
Releasable amount	Vested minus released.

10.1 Linear schedule example

Grant: 12,000 tokens. Duration: 12 months. Ignoring a cliff for the simple example, approximately 1,000 tokens become vested each month. At month 5, about 5,000 are vested; if 3,000 were already released, about 2,000 remain releasable.

Important OpenZeppelin behavior: VestingWallet can receive Ether and ERC-20 tokens and releases them according to its vesting schedule. Assets sent after vesting has already started are treated as if they had been locked from the beginning, so some of a late deposit may be immediately releasable.

11. OpenZeppelin VestingWallet Example

OpenZeppelin Contracts 5.x provides VestingWallet as a reusable finance primitive. For a normal linear schedule, the constructor receives the beneficiary, start timestamp and duration.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {VestingWallet} from
    "@openzeppelin/contracts/finance/VestingWallet.sol";

contract TeamVestingWallet is VestingWallet {
    constructor(
        address beneficiary,
        uint64 startTimestamp,
        uint64 durationSeconds
    ) VestingWallet(beneficiary, startTimestamp, durationSeconds) {}
}
```

11.1 Client design questions

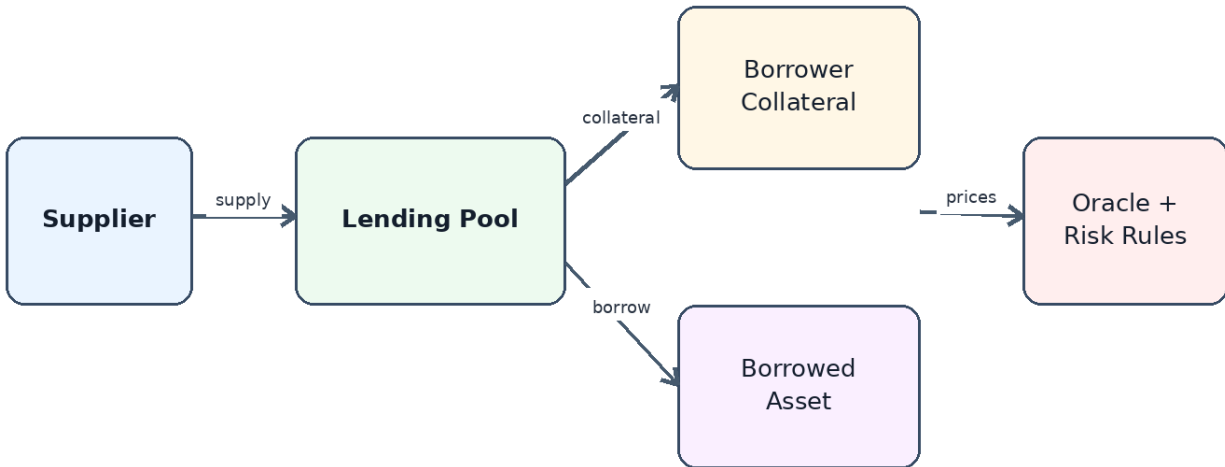
1. Linear vesting or custom curve?
2. Is there a cliff?
3. Can the beneficiary change?
4. Should unvested allocations be revocable? If yes, under what governance?
5. Which assets may be funded?
6. Who is allowed to fund the wallet?
7. How will the client report vested, released and releasable amounts in the frontend?

Security note: Do not invent revocation or beneficiary-transfer behavior casually. Those features change economic rights and require explicit product/legal requirements plus tests.

12. Lending and Borrowing Fundamentals

Collateralized DeFi lending lets suppliers provide liquidity while borrowers lock collateral and borrow another asset under protocol risk rules. Unlike an unsecured personal loan, onchain borrowing is commonly overcollateralized and continuously evaluated using oracle prices and configured thresholds.

Collateralized Lending Mental Model



Term	Developer meaning
Supply	Deposit an asset into a lending market.
Collateral	Supplied asset enabled to secure borrowing.
Borrow	Receive liquidity while creating a debt position.
LTV	Borrow capacity relative to collateral value.
Liquidation threshold	Risk boundary beyond which collateral may become liquidatable.
Health factor	Solvency indicator derived from collateral value, debt and liquidation thresholds.
Interest	Debt/supply accounting that changes over time.

Aave v3 mental model: Aave documentation describes a position as eligible for liquidation when its Health Factor falls below 1. Treat Health Factor as a risk metric, not a promise that liquidation will happen at a specific market price.

13. Health Factor - Beginner Calculation

A simplified health-factor model is:

$$\text{Health Factor} = \frac{(\text{Collateral Value} * \text{Weighted Liquidation Threshold})}{\text{Total Debt Value}}$$

Example: a user has \$10,000 of collateral, an 80% weighted liquidation threshold, and \$6,000 of debt.

$$\begin{aligned} \text{HF} &= (10,000 * 0.80) / 6,000 \\ &= 8,000 / 6,000 \\ &= 1.33 \end{aligned}$$

If collateral falls to \$7,000 while debt remains \$6,000:

$$\begin{aligned} \text{HF} &= (7,000 * 0.80) / 6,000 \\ &= 5,600 / 6,000 \\ &= 0.93 \end{aligned}$$

Interpretation: HF above 1 has buffer; HF near 1 is risky; below 1 is liquidation-eligible under the Aave v3 model. Real integrations must use protocol-provided account data and current configured parameters rather than hard-coded percentages.

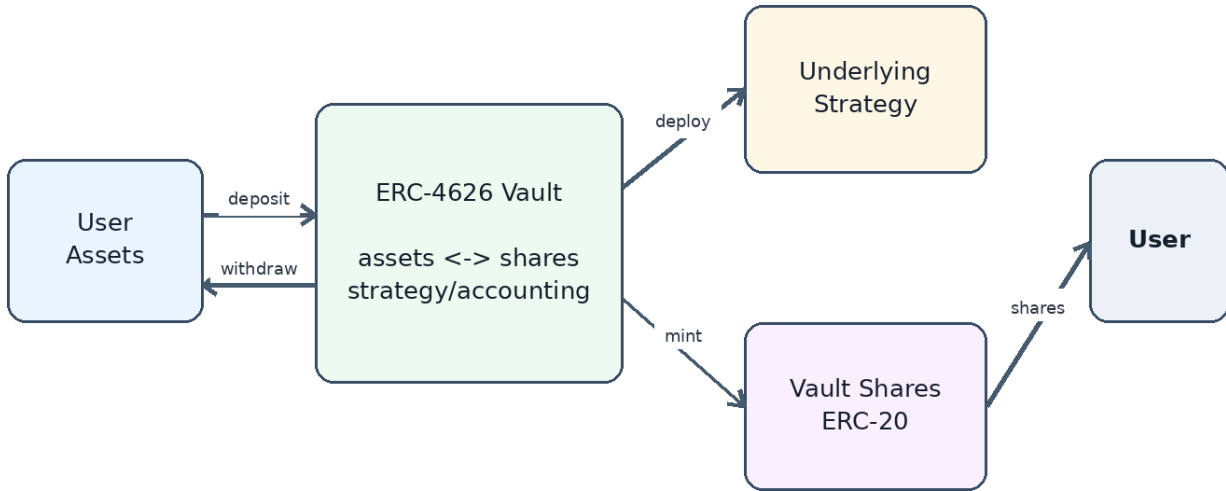
13.1 Freelancer integration tasks

- Display supplied assets, debt and health factor.
- Prevent UI actions on the wrong network or wrong market address.
- Read protocol configuration instead of copying constants from a blog post.
- Test borrow/repay/supply/withdraw flows on a fork or test environment.
- Handle allowance, token decimals, interest-bearing balances and transaction failures.

14. Yield Farming and Vaults

Yield farming is a broad term for deploying assets into one or more DeFi strategies to earn fees, incentives or interest. A vault can package that strategy behind a deposit/withdraw interface and issue shares that represent the depositor's proportional claim.

ERC-4626 Tokenized Vault Mental Model



14.1 ERC-4626 vocabulary

ERC-4626 term	Meaning
asset	The single underlying ERC-20 token accepted by the vault.
share	ERC-20 representation of a claim on vault assets.
totalAssets	Amount of underlying assets managed by the vault.
deposit/assets, receiver)	User provides assets and receives shares.
mint/shares, receiver)	User requests a specific number of shares and supplies required assets.
withdraw/assets, receiver, owner)	Redeem enough shares to receive a specific asset amount.
redeem/shares, receiver, owner)	Burn a specific number of shares and receive corresponding assets.

Standard scope: ERC-4626 standardizes a single-asset tokenized vault interface. It does not prescribe the yield strategy itself, guarantee profit, or remove slippage/rounding/security concerns.

15. ERC-4626 Share Math - Simple Example

Suppose a vault starts with 1,000 USDC-equivalent assets and 1,000 shares. The initial exchange rate is 1 asset per share. After the strategy earns 100 assets, totalAssets becomes 1,100 while shares remain 1,000.

```
Assets per share = totalAssets / totalSupply
                  = 1,100 / 1,000
                  = 1.10
```

A holder of 100 shares now represents roughly 110 assets before fees, rounding and withdrawal constraints.

15.1 Why preview functions matter

Tokenized vault integrations need predictable asset/share conversions. ERC-4626 includes conversion/preview and max functions so frontends and integrators can estimate results and limits. A production frontend should use the vault's own interface rather than assuming a fixed exchange rate.

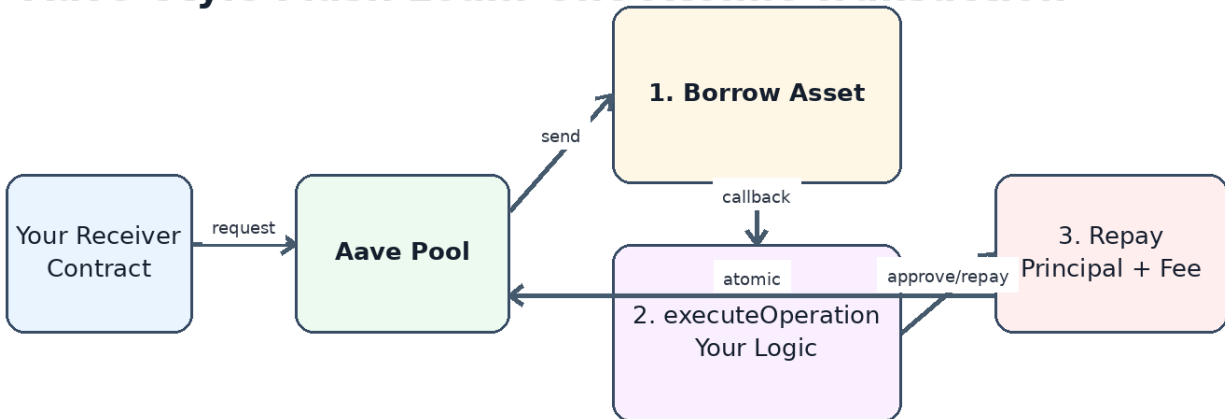
15.2 Common vault risks

- Rounding and precision errors at very small deposits.
- Donation/inflation-style share-price manipulation in poorly designed or poorly initialized vaults.
- Strategy loss or illiquidity causing totalAssets to fall or withdrawals to be limited.
- Fees changing the relationship between previews and realized amounts.
- External protocol risk inherited from the strategy.

16. Flash Loans: Atomic Liquidity

A flash loan makes liquidity available for one blockchain transaction. The receiver must satisfy repayment rules before the transaction completes; otherwise the entire transaction reverts. This atomicity is what makes flash loans different from normal collateralized borrowing.

Aave-Style Flash Loan: One Atomic Transaction



Aave v3 method	Core distinction
flashLoanSimple()	Single reserve, gas-oriented simple path; fee is not waived and it cannot finish by opening a debt position.
flashLoan()	Can borrow multiple reserves and may support ending with variable debt when requirements are met; approved flash borrowers may have fee treatment defined by protocol permissions.

Aave v3 repayment rule: The receiver must make the required amount available/approved for the Pool at the end of the operation. If repayment requirements are not met, the transaction reverts.

17. Aave-Style FlashLoanSimple Receiver Skeleton

This skeleton is for understanding the callback architecture. The business logic belongs inside `executeOperation`, but production code must validate the caller/initiator assumptions, approve repayment, handle token quirks and include strict profitability or business constraints.

```
// Teaching skeleton - addresses/network omitted
pragma solidity ^0.8.24;

import {FlashLoanSimpleReceiverBase} from
    "@aave/core-v3/contracts/flashloan/base/FlashLoanSimpleReceiverBase.sol";
import {IPoolAddressesProvider} from
    "@aave/core-v3/contracts/interfaces/IPoolAddressesProvider.sol";
import {IERC20} from
    "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from
    "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

contract FlashLoanDemo is FlashLoanSimpleReceiverBase {
    using SafeERC20 for IERC20;

    constructor(address provider)
        FlashLoanSimpleReceiverBase(IPoolAddressesProvider(provider)) {}

    function start(address asset, uint256 amount) external {
        POOL.flashLoanSimple(address(this), asset, amount, bytes(""), 0);
    }

    function executeOperation(
        address asset, uint256 amount, uint256 premium,
        address initiator, bytes calldata params
    ) external override returns (bool) {
        require(msg.sender == address(POOL), "not pool");
        require(initiator == address(this), "bad initiator");

        // Your atomic business logic goes here.

        IERC20(asset).forceApprove(address(POOL), amount + premium);
        return true;
    }
}
```

18. Flash Loan Security Checklist

Flash loans are often discussed as an attacker tool, but the risk is usually in a vulnerable protocol assumption: manipulable prices, missing access controls, unsafe accounting or temporary-liquidity assumptions. Secure applications must remain correct even when an external caller can access large temporary liquidity.

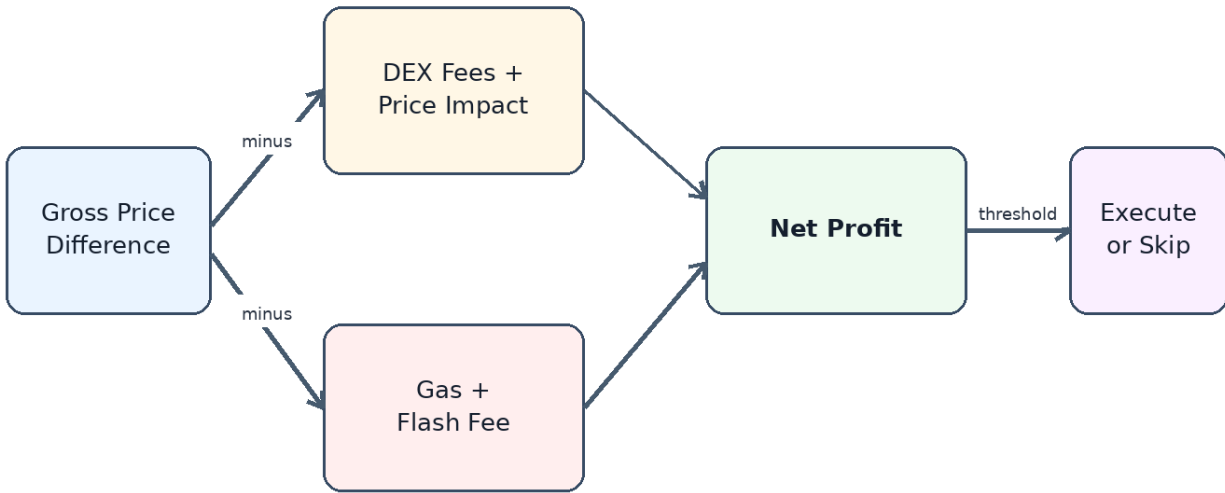
Check	Why it matters
Validate callback caller	Do not let arbitrary callers invoke privileged callback logic.
Validate initiator/params	Bind callback execution to an expected request and intent.
Use current protocol addresses	Wrong market/provider address can break or misroute integration.
Approve only required repayment	Avoid unnecessary token allowance exposure.
Never trust spot DEX price as sole oracle	Temporary liquidity can move thin pools inside one transaction.
Enforce minimum profit / max loss	Do not execute a route that is mathematically unprofitable after costs.
Use deadline/slippage protections	DEX leg execution can change between quote and inclusion.
Fork-test complete route	Mocks often miss real token/router/pool behavior.

Security principle: Ask: "If someone can borrow a very large amount for one transaction, does my protocol still enforce the same invariants?" If the answer is no, the protocol design is fragile.

19. Arbitrage: Opportunity Is Not the Same as Profit

Arbitrage attempts to exploit a temporary price difference between markets. A valid opportunity must remain profitable after every execution cost and must be executable with available liquidity and slippage constraints.

Arbitrage Profitability Is a Cost Equation



```

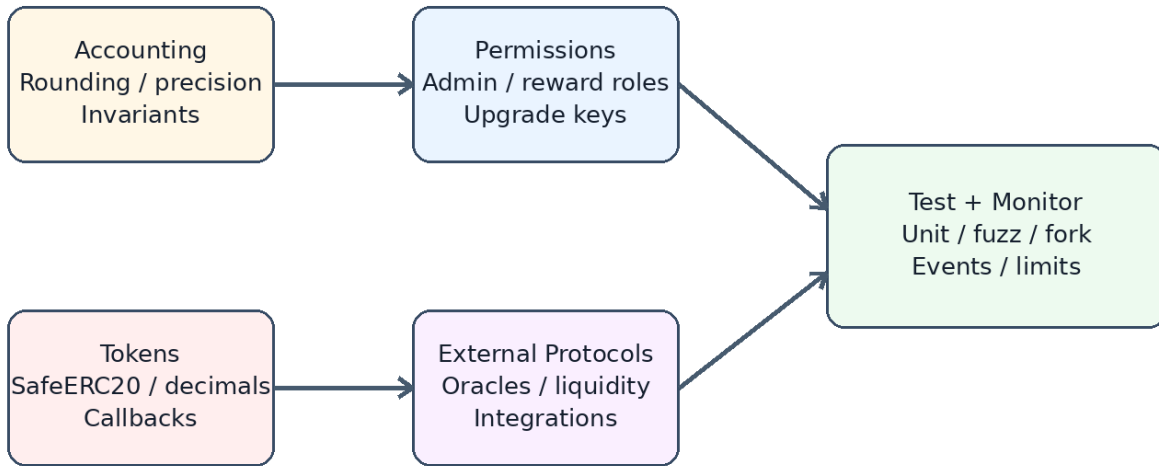
Net Profit = Gross Trading Gain
- DEX Fees
- Price Impact
- Gas Cost
- Flash Loan Fee
- Other Protocol/Bridge Costs
  
```

Example teaching numbers: buy equivalent asset on Market A for 100,000 units and sell on Market B for 100,500. Gross difference = 500. If DEX fees are 220, gas is 90, flash fee is 50 and price-impact/rounding costs are 110, the route loses money: $500 - 220 - 90 - 50 - 110 = 30$. If another 50 units of execution loss appears, net becomes -20 and the transaction should be rejected by a minimum-profit rule.

Freelancer warning: Do not promise "guaranteed arbitrage profit." Real opportunities are competitive, state changes quickly, transaction ordering matters, and simulated quotes can differ from execution. Build deterministic protection rules and observability instead of marketing guarantees.

20. Staking + Vesting + Lending: Common Security Failures

DeFi Development Security Layers



Failure pattern	Example	Test / mitigation
Reward insolvency	Contract promises more rewards than funded balance	Funding invariant; cap or validate reward schedule
Incorrect checkpoint order	New stake earns rewards from before deposit	Update reward before changing balances
Token mismatch	Assume 18 decimals or standard transfer behavior	Use token decimals carefully; SafeERC20; integration tests
Admin overreach	Owner can drain user stake	Separate user assets from recoverable admin assets; roles/timelocks
Vesting timestamp bug	Unlock earlier/later than intended	Boundary tests around start, cliff and end
Oracle manipulation	Borrow limit based on thin spot pool	Use robust oracle design and freshness/deviation checks
Vault rounding	Small depositor receives unfair shares	Property/fuzz tests on conversion math
Flash-loan callback abuse	Arbitrary caller triggers privileged logic	Validate Pool/caller, initiator and route parameters

21. Foundry Test Plan for the Staking Contract

A professional delivery includes tests that prove both normal behavior and economic boundaries. The test matrix is more valuable than a single "happy path" test.

Test	Expected result
stake(0)	Reverts.
stake(amount)	Transfers stake token and increases user/total balances.
Two stakers, different entry times	Rewards reflect time-weighted stake.
withdraw > balance	Reverts.
withdraw valid amount	Updates accounting then returns stake token.
getReward	Transfers exactly accrued reward and zeroes stored reward.
No stakers during reward period	Global accounting remains coherent; undistributed policy is understood.
Reward period overlap	Leftover reward handled according to design.
Reentrancy attempt	Protected entry point cannot be reentered.
Fuzz stake/withdraw	totalSupply equals sum of modeled user stakes.
Reward conservation	Claimed + remaining obligations do not exceed funded policy.
Token transfer failure	Transaction reverts safely; accounting does not silently corrupt.

21.1 Useful invariants

```
sum(user stake balances) == totalSupply

user stake balance never becomes negative

claimed reward cannot be claimed twice

reward accounting is monotonic during an active funded period
```

22. Hands-On Lab - Build and Explain a Client-Ready Staking Module

Use a local chain or test environment only. The goal is not to chase live yield. The goal is to prove you can implement and explain the contract lifecycle.

1. Deploy two mock ERC-20 tokens: STAKE and REWARD.
2. Deploy StakingRewards with the two token addresses.
3. Mint STAKE to Alice and Bob; mint REWARD to the owner/reward funder.
4. Approve the staking contract and have Alice stake 100 tokens.
5. Fund a 7-day reward period.
6. Advance time by one day and record Alice earned().
7. Have Bob stake 100 tokens; advance time again.
8. Compare Alice and Bob earned amounts and explain why they differ.
9. Alice claims reward, then withdraws part of her stake.
10. Write a one-page delivery note: architecture, assumptions, admin powers, events you would add, test coverage and risks.

Instructor checkpoint questions

- Why must accounting update before the stake balance changes?
- What happens while totalSupply is zero?
- What prevents the owner from promising an impossible reward rate?
- Which state variables should the frontend read to show a user dashboard?

23. Upwork / LinkedIn Client Scenarios

Client request	Professional interpretation	Key deliverables
"Build staking with 20% APY"	Clarify fixed vs variable reward, compounding assumptions, reward source and duration.	Spec, contract, tests, admin/funding model, UI calculations
"Lock team tokens for 2 years"	Clarify cliff, linear/custom schedule, beneficiary control and revocability.	Vesting architecture, deployment data, verification, release UI
"Integrate Aave lending"	Resolve network/market, assets, collateral workflow, health factor and account-data reads.	Frontend/backend integration, fork tests, error handling
"Create yield vault"	Clarify underlying asset, strategy, fees, withdrawal liquidity and share accounting.	ERC-4626 interface, strategy adapters, tests, risk disclosure
"Make flash-loan arbitrage bot"	Separate receiver contract, quote/simulation service, route constraints, private transaction concerns and profitability guard.	Architecture, local/fork tests, strict loss limits, monitoring; no profit guarantee

23.1 Scope questions before giving a price

1. Which chain and exact token contracts?
2. Is the contract upgradeable? Who controls upgrades?
3. What admin roles exist and should they be multisig/timelocked?
4. What economic formula defines rewards, fees or share conversions?
5. Which external protocols and versions are dependencies?
6. What testnet/fork environment will be used?
7. What frontend/backend deliverables are included?
8. Does the client expect an audit, or only development/testing?

24. Debugging Matrix - Common DeFi Client Problems

Symptom	Likely areas	First checks
Rewards too high/low	Checkpoint order, precision, duration, decimals	Compare rewardPerToken checkpoints and timestamps
Claim reverts	Insufficient reward balance, token behavior, pause/permissions	Contract reward balance; transfer simulation; revert data
Vesting shows zero	Start/cliff timestamp, wrong token, released amount	Read start/duration/vested/releasable state
Borrow disabled	Collateral config, LTV, health factor, asset availability	Protocol account data + reserve configuration
Vault preview differs	Fees, rounding, totalAssets changes, limits	preview/max methods at same block; strategy state
Flash loan reverts at end	Repayment/fee insufficient, approval missing, callback logic reverted	Receipt/trace; balance before repayment; allowance to Pool
Arbitrage profitable in script but loses onchain	Stale quote, price impact, gas, ordering, token taxes	Simulate exact route at current block and include all costs

Debugging order: Reproduce -> identify exact block/network/addresses -> inspect state -> simulate read-only where possible -> inspect revert/trace/logs -> isolate accounting vs token vs external-protocol issue -> add a regression test.

25. Interview Questions - Day 15

- 1. Why is looping over all stakers a problem?** Gas cost grows with user count. Scalable designs checkpoint global/user accounting on interaction.
- 2. What is rewardPerToken?** A cumulative index representing rewards allocated per unit of stake over time.
- 3. Staking vs vesting?** Staking rewards an active deposited position; vesting releases a preallocated entitlement according to time.
- 4. What is a cliff?** An initial vesting period during which no allocation is releasable.
- 5. What is health factor?** A collateral/debt solvency metric used by lending protocols; on Aave v3, below 1 is liquidation-eligible.
- 6. What does ERC-4626 standardize?** A standard interface for tokenized vault shares over a single underlying ERC-20 asset.
- 7. What makes a flash loan "flash"?** Liquidity is borrowed and repayment requirements are satisfied inside one atomic transaction or the transaction reverts.
- 8. flashLoanSimple vs flashLoan in Aave v3?** Simple handles one reserve with a simpler path; flashLoan can handle multiple reserves and has additional debt/permission behavior.
- 9. Why can a flash loan expose oracle weaknesses?** Temporary liquidity can move manipulable markets inside one transaction; protocols must not rely on fragile spot prices.
- 10. Why is gross arbitrage spread insufficient?** Fees, gas, price impact, flash premium and execution changes determine net profitability.

Short verbal drill

Prompt: Explain a staking contract to a non-technical client in 45 seconds, then explain the same system to a Solidity interviewer using reward-per-token and checkpoint terminology.

26. Day 15 Quiz

1. What three pieces of state are usually needed to compute time-weighted staking rewards?
2. Why should reward accounting update before a stake balance changes?
3. What is releasable amount in a vesting contract?
4. What does a lending protocol use collateral for?
5. What happens conceptually when health factor falls below the liquidation boundary?
6. In ERC-4626, what is the difference between assets and shares?
7. Why can share price increase even when share supply stays constant?
8. What must be true by the end of a normal flash-loan transaction?
9. Why should a receiver validate the callback caller?
10. Name four costs that reduce arbitrage profit.
11. Why is a DEX spot price dangerous as the only lending oracle?
12. Name three DeFi tests that should be fuzzed or treated as invariants.

Answer key

1. Global reward index, user checkpoint and user stake balance (plus time/rate inputs).
2. To prevent the new/old balance from receiving the wrong historical reward share.
3. Vested amount minus amount already released.
4. To secure borrowed value and absorb borrower default/price risk according to protocol rules.
5. The position becomes eligible for liquidation under the protocol model.
6. Assets are underlying ERC-20 units; shares are ERC-20 claims on vault assets.
7. Total assets can grow from strategy yield while the number of shares remains unchanged.
8. Repayment requirements including fee/premium must be satisfied, otherwise the transaction reverts.
9. To prevent arbitrary callers from invoking privileged callback behavior.
10. DEX fees, gas, price impact, flash-loan fee; also slippage/other protocol costs.
11. Temporary liquidity can manipulate a thin spot market inside a transaction.
12. Examples: stake-supply conservation, no double-claim, reward funding constraints, vault conversion monotonicity/rounding boundaries.

27. Homework and Portfolio Assignment

Homework A - Staking extension

1. Add Stake, Withdrawn, RewardPaid and RewardAdded events.
2. Add an admin pause mechanism and decide which operations should remain available during pause.
3. Write tests for two users entering at different times.
4. Fuzz stake/withdraw amounts while checking totalSupply accounting.
5. Document how reward funding remains solvent.

Homework B - Vesting design

Write a one-page specification for a 24-month team allocation with a 6-month cliff and linear vesting afterward. State whether the grant is revocable, whether beneficiary ownership can change, and how the frontend calculates releasable tokens.

Homework C - Lending analyzer

Create a small script or worksheet that accepts collateral value, weighted liquidation threshold and debt value, then calculates health factor and labels the position Healthy / Caution / Liquidation-eligible for educational purposes.

Portfolio deliverable

```
day15-defi-development/  
  contracts/StakingRewards.sol  
  test/StakingRewards.t.sol  
  notes/vesting-spec.md  
  scripts/health-factor-demo.*  
  diagrams/  
  README.md
```

README must explain:

- architecture
- reward accounting
- admin powers
- test coverage
- known assumptions/risks
- deployment is test-only unless audited

28. Day 15 Cheat Sheet

Topic	Remember
Staking	Deposit stake -> checkpoint rewards -> withdraw/claim.
Reward accounting	Global reward-per-token accumulator + user checkpoint avoids loops.
Vesting	$\text{vested}(\text{timestamp}) - \text{released} = \text{releasable}$.
Lending	Borrow capacity depends on collateral value, risk parameters and oracle prices.
Health factor	Aave v3: below 1 is liquidation-eligible.
ERC-4626	Single underlying ERC-20 asset; vault issues ERC-20 shares.
Flash loan	Atomic borrow; repay principal + premium/fee requirements in same transaction.
Arbitrage	Net profit = gross spread - every execution cost.
Security	Accounting invariants + permission boundaries + robust external integrations.
Freelancing	Clarify economics before estimating code.

Whiteboard formulas

```

rewardPerToken += elapsed * rewardRate / totalStaked

releasable = vested(timestamp) - released

healthFactor = collateralWeightedForLiquidation / debt

assetsPerShare = totalAssets / totalShares

netArb = grossSpread - dexFees - priceImpact - gas - flashFee

```

Tomorrow - Day 16

Day 16: Oracles & Blockchain Automation - Chainlink price feeds, freshness/decimals checks, Automation concepts, offchain/onchain data boundaries and a price-aware smart-contract lab.

29. Primary Technical References

Use primary documentation when implementing client work. Protocol interfaces, contract packages, addresses and risk parameters can change. Re-check the target network and exact deployed version before deployment.

Source	What to review
Aave Protocol Documentation - Aave v3 Overview	Supply/borrow model, position health and liquidation concepts.
Aave Protocol Documentation - Flash Loans	flashLoan() vs flashLoanSimple(), atomic repayment and receiver workflow.
Aave Protocol Documentation - Pool	Pool flash-loan functions, repayment/approval behavior and integration warnings.
OpenZeppelin Contracts 5.x - Finance / VestingWallet	Beneficiary ownership, vesting schedule, releasable/released behavior for native and ERC-20 assets.
ERC-4626: Tokenized Vaults	Standard assets/shares/deposit/mint/withdraw/redeem interface and security considerations.
OpenZeppelin Contracts 5.x - SafeERC20 / ReentrancyGuard / Access	Reusable primitives used in professional token-moving contracts.

Version discipline: This lecture uses Solidity 0.8.x style, OpenZeppelin Contracts 5.x conventions, and Aave v3 flash-loan interfaces for the teaching examples. For real jobs, pin dependency versions and verify deployed protocol addresses from official sources.

Day 15 Completion Standard

You are ready for Day 16 when you can explain the staking accumulator without a loop, design a vesting schedule, calculate a simple health factor, describe assets vs shares in ERC-4626, trace a flashLoanSimple callback, and explain why a price difference is not automatically profitable arbitrage.